



TRAINING PROGRAMME



Lesson 1

Introduction to Refactoring

*"Refactor not because you know the right abstraction,
but because you want to find it."— Martin Fowler*

IT'S ALL GREEN...

...then REFACTOR!!!

Refactor ***aggressively*** and ***constantly***

WHEN?



REFACTORING

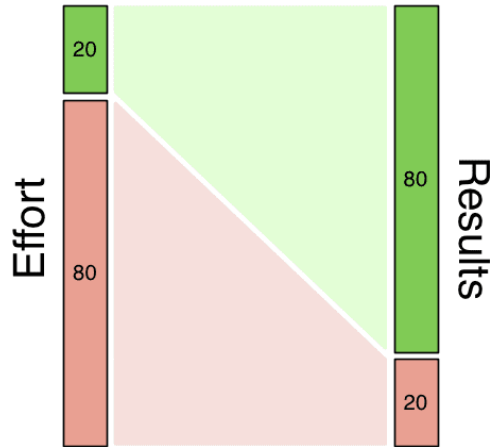
- When we find duplication (Rule of )
- When we break Object Calisthenics rules.



THE PARETO PRINCIPLE

The 80-20 Rule

"For many events, roughly 80% of the effects come from 20% of the causes." - Pareto



**20% of actions
will account for
80% of results.**

Therefore 20% of the effort produces 80% of the results but the last 20% of the results consumes 80% of the effort.

www.EndlesslyCurious.com



Lesson 1

TRAINING PROGRAMME

REFACTORING 80-20 RULE

80% of the value in refactoring comes
from ***improving readability.***

20% of the remaining value comes
from ***design change.***



IDE agility: know your shortcuts

The 5 main atomic refactors of your IDE

- **Rename** – Change the name of classes, methods, variables....
- **Extract** – Extract a class (or methods or variables...), creating a new abstraction.
- **Inline** – The inverse of extract – inline a method (or variable), deconstructing an abstraction.
- **Move** – Move a class (or methods or variables...) to some other place in the codebase.
- **Safe delete** – Delete code and its usages in the code base.



Refactoring guidelines

STAY IN GREEN WHILE REFACTORING

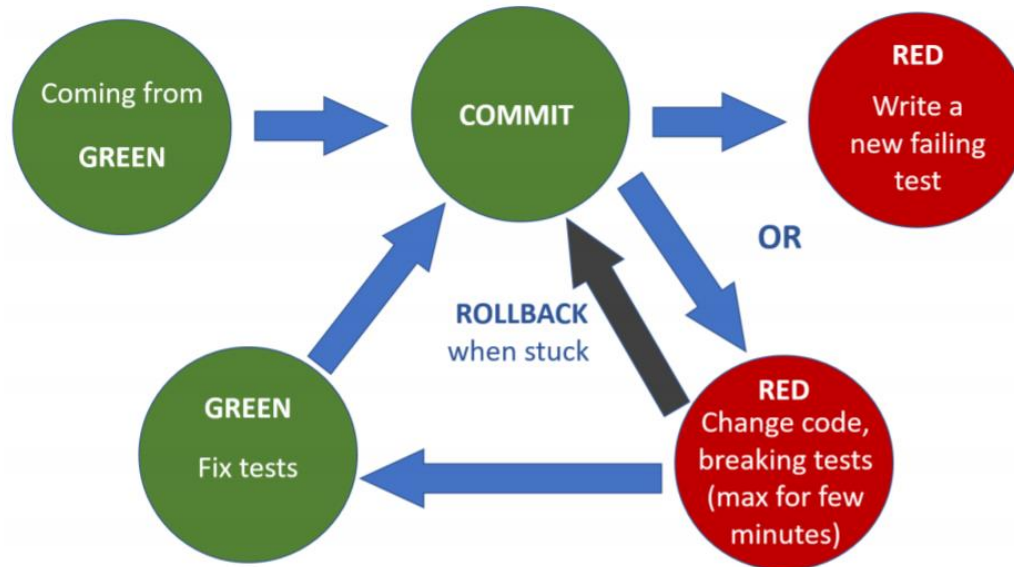
There is no reason why we should break any tests during refactoring. It may be that we have tests, but they are coupled with implementation. In this case, we start refactoring by the tests, decoupling the test from the implementation.

Be strict about staying on green. We learned that in refactoring, it is more effective to let go of something as soon as tests break, *rather than stubbornly trying to fix things* to make tests pass.



Refactoring guidelines

COMMIT AS OFTEN AS POSSIBLE



Extended TDD cycle: the Refactor cycle



Refactoring guidelines

REFACTOR READABILITY BEFORE DESIGN

Small improvements in *code readability* can drastically improve *code understandability*.

Start with *better names* for variables, methods and classes.

The idea is to **express intent** rather than implementation details. We recommend Arlo Belshee's approach for naming.

<https://www.digdeeproots.com/articles/on/naming-process/>



REFACTOR READABILITY BEFORE DESIGN

Format

→Format consistently and don't force the reader to waste time fixing formatting.

Rename

→Rename bad names, variables, arguments, instance variables, methods, classes →Make abbreviations explicit.

Remove

→Delete unnecessary comments →Delete dead code. Don't make the reader waste time trying to figure out code that is not in use anymore.

Extract

→Constants from magic numbers and strings →Conditionals.

Reorder

→Refine scope for improper variable scoping, and make sure variables are declared close to where they are used.



THEN REFACTOR THE DESIGN (simple changes)

- Extract private methods from deep conditionals.
- Extract smaller private methods from long methods
- Encapsulate cryptic code in private methods
- Return from methods as soon as possible.
- Encapsulate where we find missing encapsulation.
- Remove duplication.



Parallel change (or *expand, migrate and contract*)

It's a refactor technique to implement
breaking changes safely
(**staying on the green**).

It consists of three steps:
1-expand, 2-migrate and 3-contract.



1 - *Expand*

Introduce new functionality by adding new code instead of changing existing code.

Starting from the test, either by writing a new one or by duplicating an existing one.





TRAINING PROGRAMME



Lesson 1

Introduction to Refactoring

*"Refactor not because you know the right abstraction,
but because you want to find it."— Martin Fowler*

IT'S ALL GREEN...

...then REFACTOR!!!

Refactor ***aggressively*** and ***constantly***

WHEN?



REFACTORING

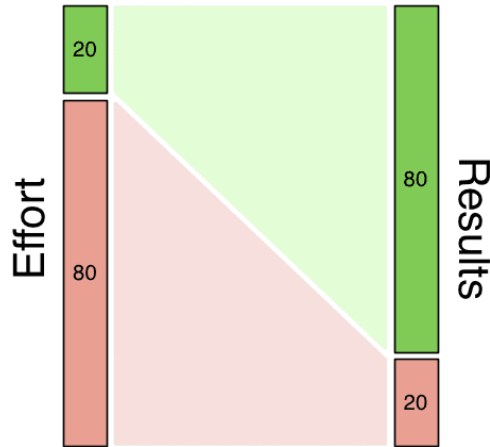
- When we find duplication (Rule of )
- When we break Object Calisthenics rules.



THE PARETO PRINCIPLE

The 80-20 Rule

"For many events, roughly 80% of the effects come from 20% of the causes." - Pareto



**20% of actions
will account for
80% of results.**

Therefore 20% of the effort produces 80% of the results but the last 20% of the results consumes 80% of the effort.

www.EndlesslyCurious.com



Lesson 1

TRAINING PROGRAMME

REFACTORING 80-20 RULE

80% of the value in refactoring comes
from ***improving readability.***

20% of the remaining value comes
from ***design change.***



IDE agility: know your shortcuts

The 5 main atomic refactors of your IDE

- **Rename** – Change the name of classes, methods, variables....
- **Extract** – Extract a class (or methods or variables...), creating a new abstraction.
- **Inline** – The inverse of extract – inline a method (or variable), deconstructing an abstraction.
- **Move** – Move a class (or methods or variables...) to some other place in the codebase.
- **Safe delete** – Delete code and its usages in the code base.



Refactoring guidelines

STAY IN GREEN WHILE REFACTORING

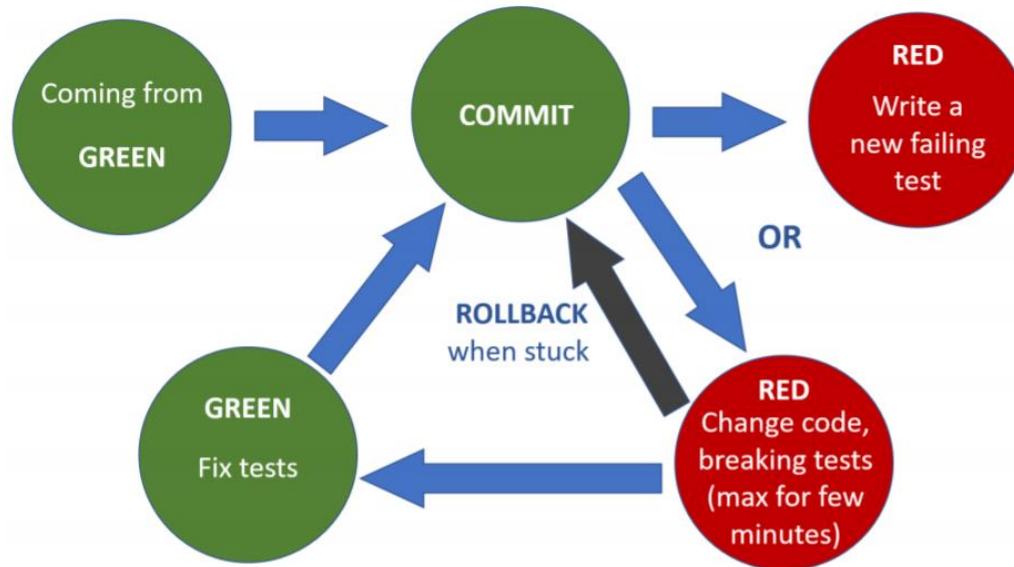
There is no reason why we should break any tests during refactoring. It may be that we have tests, but they are coupled with implementation. In this case, we start refactoring by the tests, decoupling the test from the implementation.

Be strict about staying on green. We learned that in refactoring, it is more effective to let go of something as soon as tests break, *rather than stubbornly trying to fix things* to make tests pass.



Refactoring guidelines

COMMIT AS OFTEN AS POSSIBLE



Extended TDD cycle: the Refactor cycle



Refactoring guidelines

REFACTOR READABILITY BEFORE DESIGN

Small improvements in *code readability* can drastically improve *code understandability*.

Start with *better names* for variables, methods and classes.

The idea is to **express intent** rather than implementation details. We recommend Arlo Belshee's approach for naming.

<https://www.digdeeproots.com/articles/on/naming-process/>



REFACTOR READABILITY BEFORE DESIGN

Format

→Format consistently and don't force the reader to waste time fixing formatting.

Rename

→Rename bad names, variables, arguments, instance variables, methods, classes →Make abbreviations explicit.

Remove

→Delete unnecessary comments →Delete dead code. Don't make the reader waste time trying to figure out code that is not in use anymore.

Extract

→Constants from magic numbers and strings →Conditionals.

Reorder

→Refine scope for improper variable scoping, and make sure variables are declared close to where they are used.



THEN REFACTOR THE DESIGN (simple changes)

- Extract private methods from deep conditionals.
- Extract smaller private methods from long methods
- Encapsulate cryptic code in private methods
- Return from methods as soon as possible.
- Encapsulate where we find missing encapsulation.
- Remove duplication.



Parallel change (or *expand, migrate and contract*)

It's a refactor technique to implement
breaking changes safely
(**staying on the green**).

It consists of three steps:
1-expand, 2-migrate and 3-contract.



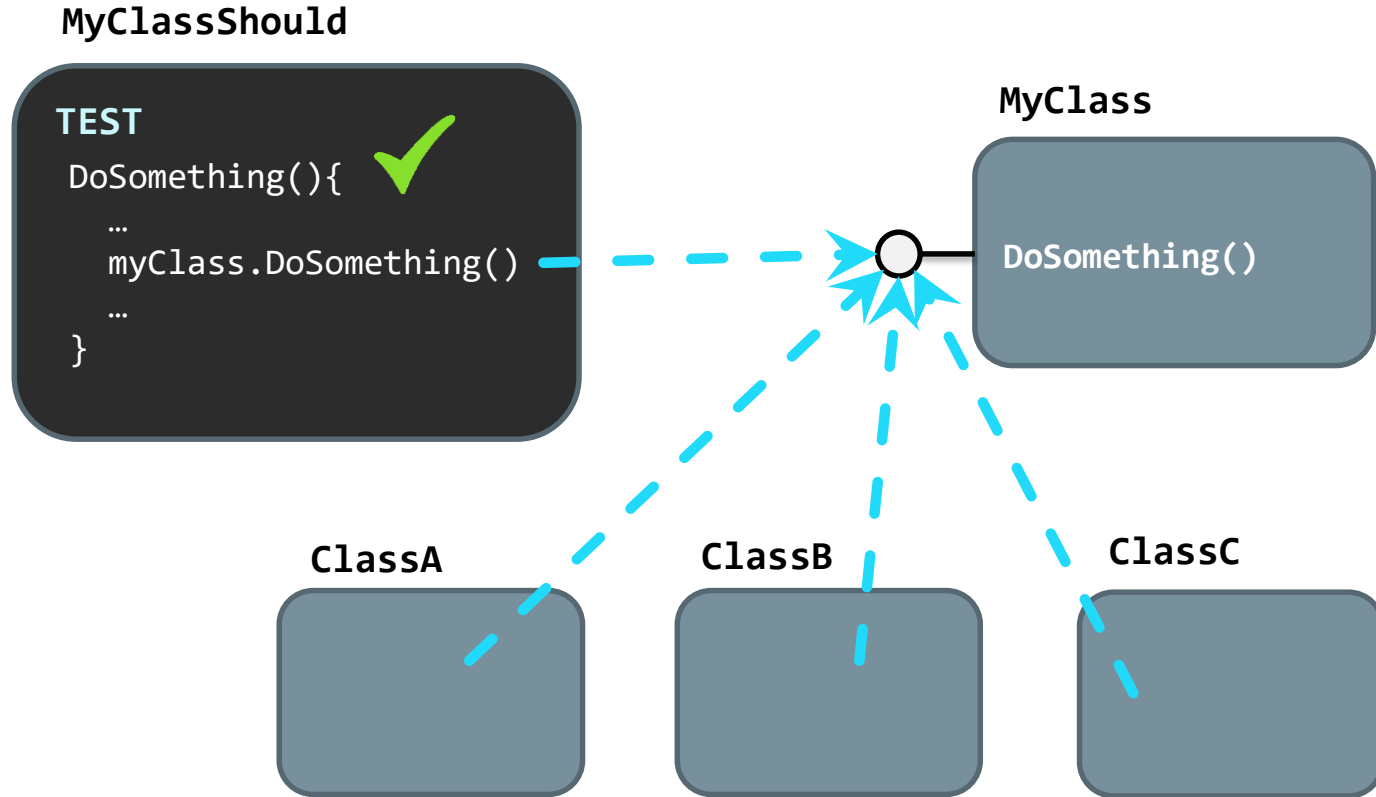
1 - *Expand*

Introduce new functionality by adding new code instead of changing existing code.

Starting from the test, either by writing a new one or by duplicating an existing one.



1 - Expand



1 - Expand

MyClassShould

TEST

```
DoSomething(){ ✓  
  ...  
  myClass.DoSomething()  
  ...  
}  
  
DoSomethingBetter(){ ✓  
  ...  
  myClass.DoSomething()  
  ...  
}
```

MyClass

DoSomething(){...}

ClassA

ClassB

ClassC



Lesson 1

TRAINING PROGRAMME

1 - Expand

MyClassShould

TEST

```
DoSomething(){  
  ...  
  myClass.DoSomething()  
  ...  
}
```

```
DoSomethingBetter(){  
  ...  
  myClass.DoSomethingBetter()  
  ...  
}
```

ClassA

ClassB

ClassC

MyClass

DoSomething(){...}

DoSomethingBetter(){
 Throw;
}



1 - Expand

MyClassShould

TEST

```
DoSomething(){
```



```
...  
myClass.DoSomething()  
...
```

```
}
```

```
DoSomethingBetter(){
```



```
...  
myClass.DoSomethingBetter()  
...
```

```
}
```

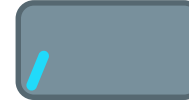
ClassA



ClassB



ClassC



MyClass

```
DoSomething(){...}
```

```
DoSomethingBetter(){  
...  
}
```



Lesson 1

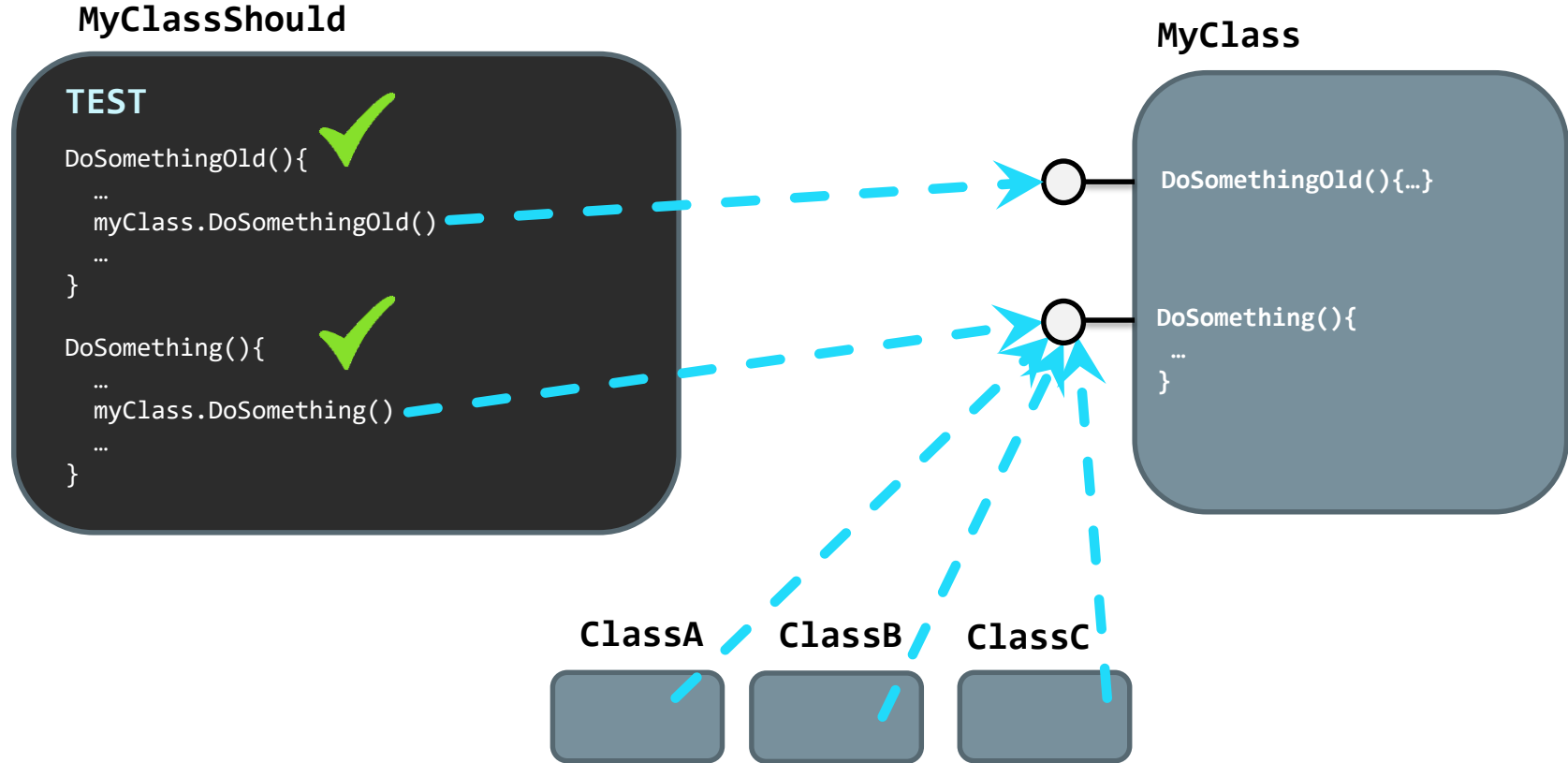
TRAINING PROGRAMME

2 - Migrate

Deprecate old code and allow clients to migrate to new expanded code, or change client code to point to new code.



2 - Migrate

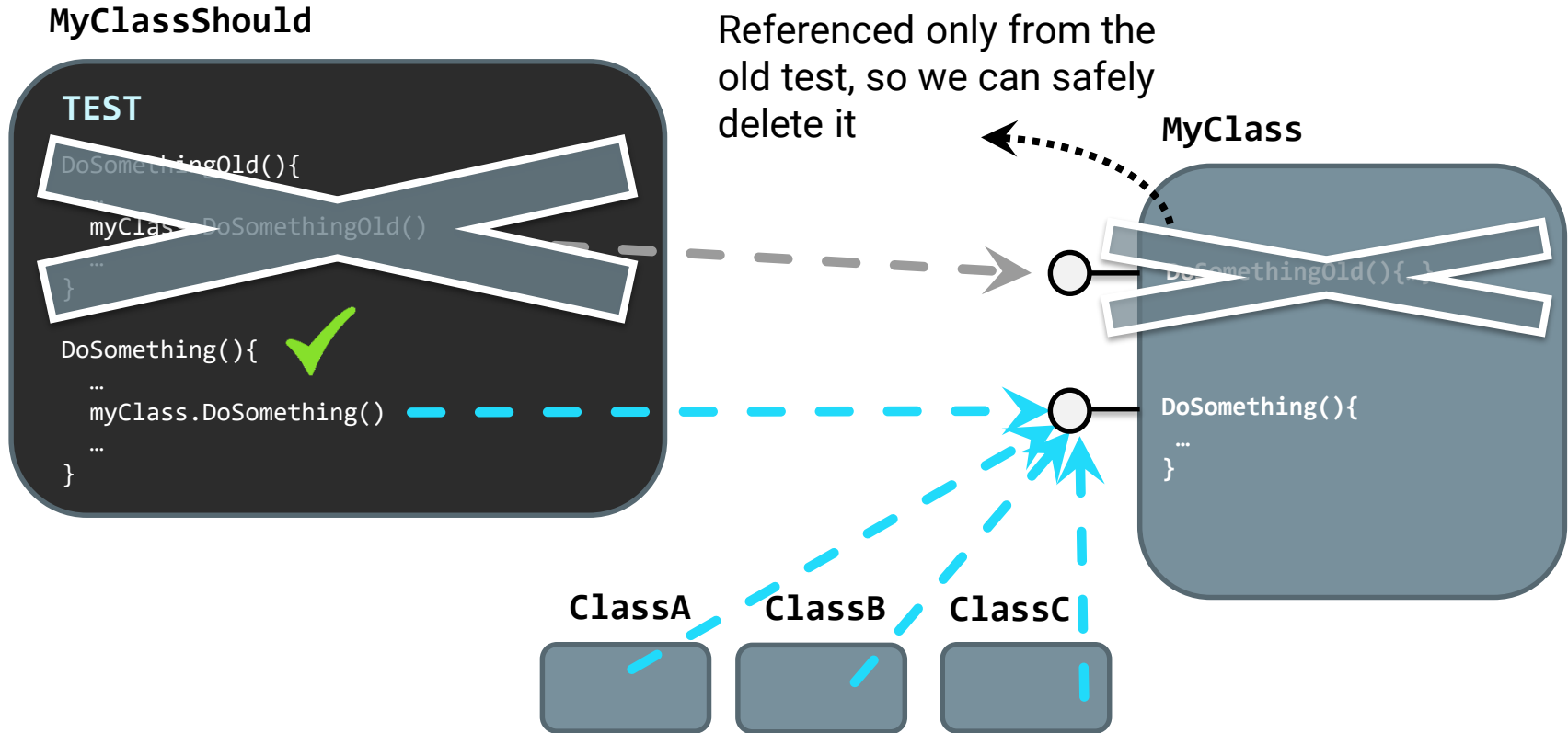


3 - Contract

Once all client code is migrated to new code, remove old functionality by removing deprecated code and its tests.



3 - Contract



KATA



Tennis refactoring by Emily Bache

Clean-up the code to a point where someone can read it and understand it with ease. You can find the repository here:

<https://github.com/emilybache/Tennis-Refactoring-Kata/tree/master/typescript>



KATA



Gilded Rose refactoring by Emily Bache

You can find the requirements here:

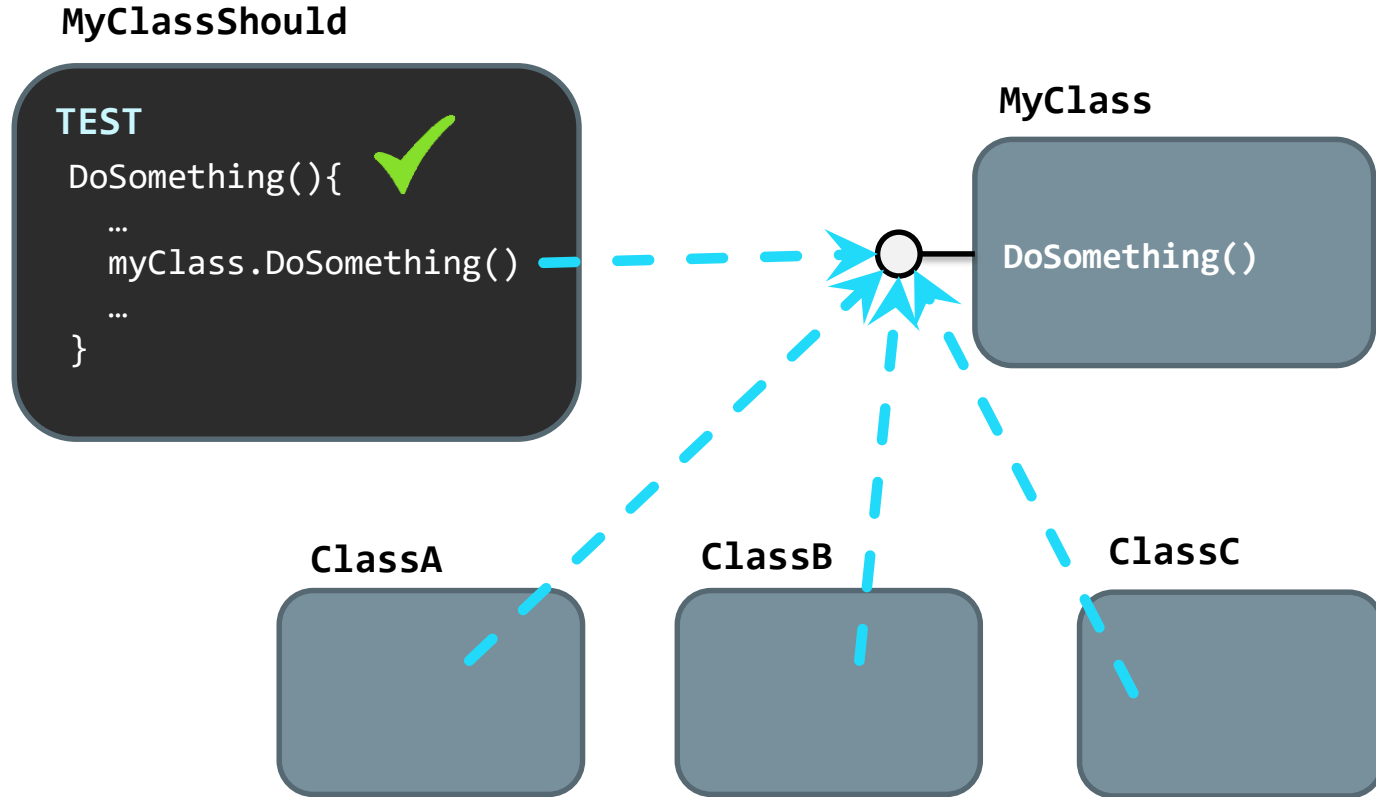
<https://github.com/emilybache/GildedRose-Refactoring-Kata/blob/master/GildedRoseRequirements.txt>

You can find the Typescript repository here:

<https://github.com/emilybache/Tennis-Refactoring-Kata/tree/master/typescript>



1 - Expand



1 - Expand

MyClassShould

TEST

```
DoSomething(){ ✓  
  ...  
  myClass.DoSomething()  
  ...  
}  
  
DoSomethingBetter(){ ✓  
  ...  
  myClass.DoSomething()  
  ...  
}
```

MyClass

DoSomething(){...}

ClassA

ClassB

ClassC



Lesson 1

TRAINING PROGRAMME

1 - Expand

MyClassShould

TEST

```
DoSomething(){  
    ...  
    myClass.DoSomething()  
    ...  
}
```

```
DoSomethingBetter(){  
    ...  
    myClass.DoSomethingBetter()  
    ...  
}
```

ClassA

ClassB

ClassC

MyClass

DoSomething(){...}

DoSomethingBetter(){
 Throw;
}



1 - Expand

MyClassShould

TEST

```
DoSomething(){
```



```
...  
myClass.DoSomething()  
...
```

```
}
```

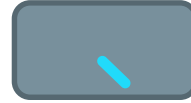
```
DoSomethingBetter(){
```



```
...  
myClass.DoSomethingBetter()  
...
```

```
}
```

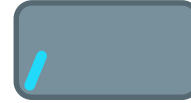
ClassA



ClassB



ClassC



MyClass

```
DoSomething(){...}
```

```
DoSomethingBetter(){  
...  
}
```



Lesson 1

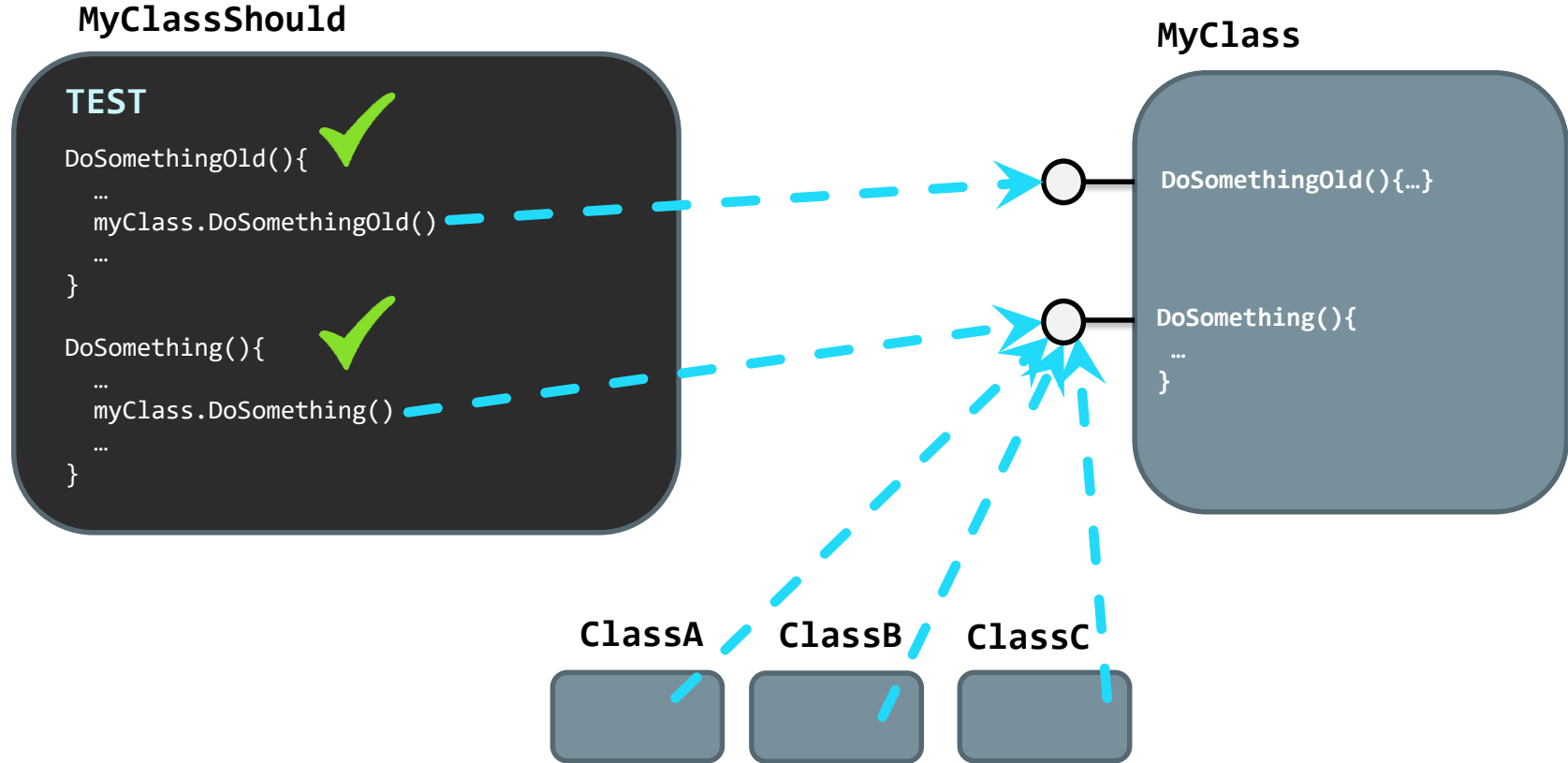
TRAINING PROGRAMME

2 - Migrate

Deprecate old code and allow clients to migrate to new expanded code, or change client code to point to new code.



2 - Migrate

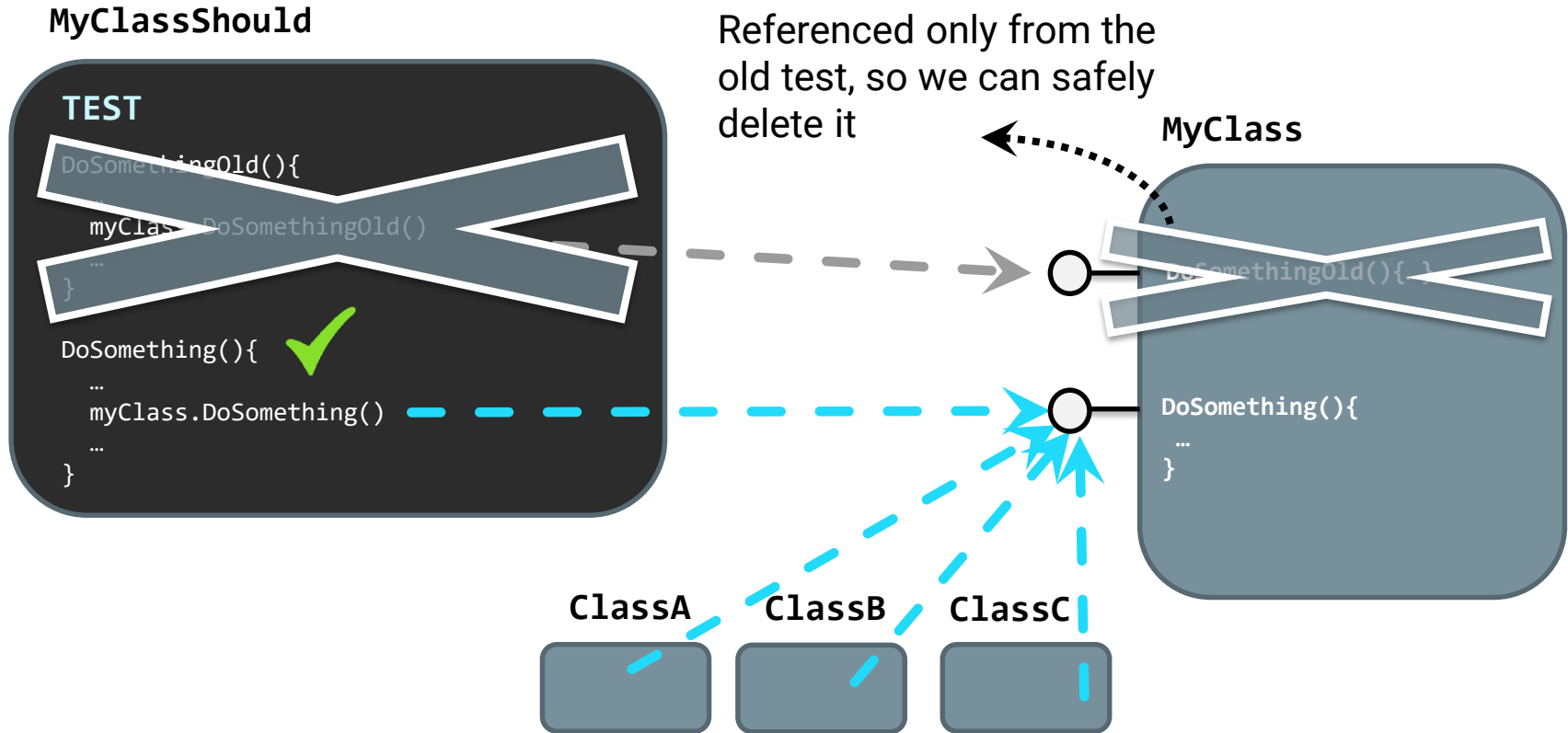


3 - Contract

Once all client code is migrated to new code, remove old functionality by removing deprecated code and its tests.



3 - Contract



KATA



Tennis refactoring by Emily Bache

Clean-up the code to a point where someone can read it and understand it with ease. You can find the repository here:

<https://github.com/emilybache/Tennis-Refactoring-Kata/tree/master/typescript>



KATA



Gilded Rose refactoring by Emily Bache

You can find the requirements here:

<https://github.com/emilybache/GildedRose-Refactoring-Kata/blob/master/GildedRoseRequirements.txt>

You can find the Typescript repository here:

<https://github.com/emilybache/Tennis-Refactoring-Kata/tree/master/typescript>

